

Variables de entorno en GNU/Linux

Gonzalo F. Perez Paina

Marzo de 2025

1. Variables de la Shell

Una Shell en ejecución puede definir variables y almacenar valores en ellas. Un ejemplo de una variable de la Shell es `HOME`, cuyo valor es la ruta (path) al directorio de trabajo o “home”, tal como `/home/infoii`. Otro ejemplo es `USER`, cuyo valor es el nombre del usuario que ejecuta el comando. Una forma de mostrar el valor de las variables `HOME` y `USER` en la salida estándar, es mediante la ejecución del comando `printenv`:

```
$> printenv HOME
/home/infoii
$> printenv USER
infoii
```

Por otro lado, la Shell puede evaluar una variable; o sea, reemplazar el nombre de la variable con su valor. Para evaluar una variable hay que anteponer el signo `$` al nombre de la variable; por ejemplo, `$HOME` se evalúa como la cadena `/home/infoii`. La forma más sencilla de observar el valor de una variable en la línea de comando es mediante el comando `echo` (el cual imprime su argumento):

```
$> echo "Mi nombre es $USER y mis archivos están en $HOME"
Mi nombre es infoii y mis archivos están en /home/infoii
```

Las variables tales como `USER` y `HOME` están predefinidas en la Shell, sus valores se fijan de forma automática cuando se inicia la sesión de trabajo. Generalmente, estas variables tienen sus nombres en mayúsculas. Es posible también definir variables propias del usuario o modificar sus valores utilizando la sintaxis:

name=value

Por ejemplo, si se trabaja frecuentemente en el directorio `/home/infoii/programas`, se puede asignar esta ruta a una variable:

```
$> mis_prog=$HOME/programas
```

y utilizar la variable como un atajo con el comando `cd`, como:

```
$> cd $mi_prog
$> pwd
/home/infoii/programas
```

O bien, utilizar `$mis_prog` en cualquier comando que espera un directorio, como:

```
$> cp main.c $mis_prog
$> ls $mis_prog
main.c
```

Al definir una variable, no hay que dejar espacios entre el signo `=`. Si se dejan los espacios, la Shell asume que la primer palabra es un comando:

```
$> mis_prog = $HOME/programas
mis_prog: command not found
```

2. Programa de la Shell (procesos padre e hijo)

El propósito de una Shell, el cual consiste en ejecutar comandos, resulta tan básico en todo sistema GNU/Linux que se puede llegar a pensar que la Shell están construida dentro del propio SO GNU/Linux, lo cual no es cierto; una Shell es un programa común tal como lo son **echo** y **cat**. Este programa repite de forma continuada los siguientes pasos: 1) muestra/imprime el prompt, 2) lee el comando desde la entrada estándar, 3) evalúa y ejecuta el comando e 4) imprime el resultado del comando en la salida estándar. La Shell por defecto en muchas distribuciones del SO GNU/Linux es **bash**. Para ver cuál es la Shell en ejecución:

```
$> echo $SHELL
```

Dado que **bash** es un programa, se puede ejecutar de forma manual como cualquier otro comando:

```
$> bash
```

Para ver el cambio de Shell se puede modificar el “prompt” modificando el valor de la variable **PS1**:

```
$> PS1="PromptInfoII: "
```

Luego, se puede ejecutar el comando **exit** para salir de la nueva instancia de **bash**. Se puede determinar cuántas sub-shells están abiertas utilizando la variable de entorno **SHLVL**:

```
$> echo $SHLVL
1
$> bash
$> echo $SHLVL
2
$> exit
$> echo $SHLVL
1
```

3. Algunos comentarios sobre procesos padre e hijo

Cuando una instancia de la Shell invoca a otra, la Shell original se llama “padre” y la nueva se llama “hijo”. Un programa en ejecución en un SO GNU/Linux se conoce como “proceso” por lo que también se puede nombrar como “proceso padre” y “proceso hijo”. Un proceso puede invocar a varios hijos, pero un hijo tiene un único padre.

Cada proceso tiene su “entorno”, el cual incluye el directorio actual (o directorio de trabajo), rutas de búsqueda, prompt de la Shell y demás información de relevancia que se almacena en las variables de la Shell. Cuando se crea un proceso hijo, su entorno es en gran medida una copia del padre. Cada vez que se ejecuta un comando simple, se está creando un proceso hijo. O sea, que aún cuando se ejecuta un comando simple como **ls**, este comando se está ejecutando en un proceso hijo con su propia copia del entorno.

Probar el siguiente script de la Shell (**\$>bash testcd**):

```
1 cd /etc
2 echo "El directorio actual es:"
3 pwd
```

Un comando combinado (que hace uso de tuberías) ejecuta múltiples procesos hijos.

4. Variables de entorno

Cada instancia de una Shell tiene definidas una serie de variables, algunas de las cuales les pertenece a esa única Shell, llamadas “variables locales”. Otras variables se copian de forma automática desde una Shell a todos sus procesos hijos: estas variables reciben el nombre de “variables de entorno”. Todas estas variables conforman el entorno de la Shell. Algunos ejemplos de variables de entorno y su uso son:

- **HOME**: Ruta al directorio “home” del usuario. Su valor se fija de forma automática cuando se inicia sesión.
- **USER**: Nombre del usuario.
- **PATH**: Rutas de búsqueda para los comandos de la Shell. Es una lista de directorios separados por “:” (dos puntos).
- **PWD**: Ruta al directorio actual de la Shell. Su valor se actualiza de forma automática por la Shell cada vez que se cambia de directorio con el comando `cd`.
- **EDITOR**: Nombre del editor predefinido. Su valor se fija generalmente en un archivo de configuración de la Shell.
- **PS1**: Cadena para representar el prompt primario.
- **SHLLV**: Se incrementa en uno cada vez que se inicia una instancia de la Shell.
- **?**: Almacena el valor de salida/retorno del último comando que se ejecutó.

Para ver todas las variables de la Shell, se puede utilizar el comando:

```
$> printenv | sort -i | less
```

Las variables locales no se muestra mediante el comando `printenv`:

```
$> materia="Informática II"
$> echo $materia
Informática II
$> printenv materia
```

Para convertir una variable local en una variable de entorno, se utiliza el comando `export`, como:

```
$> VAR1=10
$> export VAR1
$> export VAR2=20
```

El comando `export` indica que la variable junto a su valor sean copiados desde la Shell actual a cualquier proceso hijo futuro. Las variables locales no se copian a los procesos hijos.

```
$> export ENV_VAR="Variable de entorno"
$> LOC_VAR="Variable local"
$> echo $ENV_VAR
Variable de entorno
$> echo $LOC_VAR
Variable local
$> bash
$> echo $ENV_VAR
Variable de entorno
$> echo $LOC_VAR
Variable local
$> exit
```

Es posible modificar el valor de una variable copiada a un proceso hijo, pero este cambio no afecta a la variable en la Shell padre.

5. Configuración del entorno

Cuando se ejecuta una Shell como **bash**, este se configura a partir de la lectura de una secuencia de archivos conocidos como “archivos de configuración”. Estos archivos definen variables, alias, funciones y otras características de la Shell, y pueden incluir comandos comunes (son como scripts de la Shell que configuran la propia Shell).

Algunos archivos de configuración están definidos por el administrador del sistema y se aplican a todos los usuarios; generalmente se encuentran en el directorio **/etc**. Algunos otros archivos de configuración son propiedad de cada usuarios de forma individual, por lo que puede ser modificados por ellos mismos; estos se encuentran en el directorio del usuario.

Algunos de los archivos de configuración son: **/etc/profile**, **/etc/bash.bashrc**, **\$HOME/profile** y **\$HOME/.bashrc**.

Al modificar un archivo de configuración, se puede forzar a que la Shell lo lea nuevamente:

```
$> source ~/.bashrc
```

el cual utiliza el comando (builtin) **source**, o bien:

```
$> . ~/.bashrc
```

que utiliza el operador “punto”.